

Cabbage 3: Not Your Garden Variety Vegetable

Rory Walsh¹ *

Dundalk Institute of technology, Louth, Ireland rory.walsh@dkit.ie

Abstract. In an ever-evolving AI landscape, rising subscription costs for proprietary models like Claude and GitHub Copilot threaten to price out small-scale indie developers. To bypass these financial barriers, this paper explores how the Cabbage 3 ecosystem leverages JavaScript to accelerate frontend development. By combining Csound’s high-performance audio processing with decoupled, web-driven frontend architectures, Cabbage 3 provides a highly structured framework for AI-assisted development. Its predictable JSON layout engine, custom-widget boilerplates, and minimalist two-way JavaScript API natively lend themselves to high-accuracy code generation by open-source LLMs without triggering token-heavy hallucinations. Ultimately, this paper argues that Cabbage 3 remains deeply relevant precisely because its structural design bridges the gap between low-friction, AI-assisted web design and human-driven, high-performance DSP.

Keywords: Csound, Cabbage, DSP, AI, Gen-AI, Synthesis, Audio Plugins, CLAP/VST/AU, JavaScript, CSS, HTML

1 Introduction

After years of hooking users into using their models, large AI tech companies are now looking to recap those initial costs by forcing users toward higher tier subscription costs. When first launched, tools like ChatGPT and Microsoft Copilot heavily subsidized new users, but now they are aggressively transitioning from “user acquisition mode” to “monetization mode”[1]. On June 1st this year, Microsoft moved to usage billing for all models in its GitHub Copilot AI assistant. In doing so they also removed all free tier models and agents. Although Anthropic has maintained its free \$20 a month entry level subscription, users have reported more and more problems with it. Rather than address these issues Anthropic has introduced new ‘pro’ tiers starting at \$100 a month. The most frustrating thing about all of this for indie developers is just how good these AI tools have become. And as they get better with each iteration, so is the temptation to keep using them, even if it means coughing up monthly subscriptions.

Cabbage 3 is uniquely positioned to streamline development and release cycles for audio developers, whilst avoiding the costs of pro-tier AI models. By utilising WebView UIs, the entire frontend is built using vanilla JavaScript. Because open-source AI models like DeepSeek, Qwen 3, MiniMax, GLM, and Kimi are highly optimised for web languages, they excel at generating routine JS code with high accuracy, something they struggle to do with lower-level C/C++.[8] This allows developers to effortlessly offload their UI creation to cost-effective AI assistants, keeping development fast, agile, and budget-friendly.

This paper explores strategies for leveraging open-source, cost-effective AI models to streamline UI development for Cabbage 3 plugins. It begins by focusing on the standard UI components included with Cabbage 3, then outlines the architectural pathways for how users can seamlessly integrate custom components to function as first-class citizens within Cabbage’s frontend ecosystem. This approach offers developers a means of minimising costs while maximizing efficiency across the development lifecycle. Note the author does not give any consideration to the use of AI in generating Csound code. This paper’s sole focus is on outsourcing rapid UI development to low-cost, and open source AI models, so developers can remain focused on the audio backend.

2 The Cabbage 3 Ecosystem

By integrating modern web technologies with the power of Csound[2], the Cabbage 3 ecosystem represents a significant change from its predecessors. By utilising the CLAP (CLever Audio Plugin)

* Victor Lazzarini, Steven Yi

standard as its core wrapper, it ensures broad compatibility and high performance across contemporary DAWs, while also embracing a new open source format. At its heart, the ecosystem continues to leverage Csound 7 for audio processing, while adopting webviews to handle the user interface. This architectural transition, away from the limited native rendering of previous iterations of Cabbage, empowers developers to build sophisticated, responsive, and hardware-accelerated GUIs using standard HTML5, CSS, and JavaScript. In doing so, the aesthetic design is largely decoupled from the underlying audio logic.

2.1 Frontend and Backend Communication

The channel system from Cabbage 2 has been significantly extended to allow for much more efficient data transfer. Instead of relying on a simple, unique identifier for each individual attribute, Cabbage 3 employs JSON for all messaging. These messages are sent and received using two distinct mechanisms:

- **Channel-Only Updates:** The `cabbageGetValue` and `cabbageSetValue` opcodes update channel values only. This approach represents the quickest and most efficient way to update any Csound channels.
- **Attribute Updates:** The `cabbageGet` and `cabbageSet` opcodes handle larger, more complex JSON messages. These are used to update and retrieve all other attributes of a frontend element.

Because these JSON messages often contain nested elements, specific attributes are accessed using standard dot notation, as demonstrated in the example code below.

```
trig:k = metro(10)
cabbageSet(trig, "gainSlider", "bounds.left", abs(randi:k(400, 10)))
```

2.2 The Message Queue

Halfway through its lifecycle, Cabbage 2 introduced a queue mechanism for UI updates. While this represented a vast improvement over the previous polling system, it still relied on a timer and was never entirely thread-safe. Cabbage 3 addresses this by using a multi-producer, multi-consumer lock-free concurrent queue[4].¹ This FIFO queue not only ensures thread safety but also manages data redundancy, ensuring that only updated objects are passed between the frontend and backend.

Needless to say, one should continue to exercise care when passing messages back and forth. All messages are transmitted as strings.² Consequently, it is easy to clog the message queue, particularly when handling large arrays. Finally, it should be noted that Cabbage maintains a stateful backend/stateless frontend architecture. All JSON objects are managed by the backend; updates made to these objects in the frontend are immediately registered by the backend through a series of C++/JS communication hooks.

2.3 WebView wonder, or the wonder of view?

There is ongoing debate regarding the suitability of webviews for audio plugins[5]. Beyond the critical need to manage thread-to-thread communication to avoid performance bottlenecks, developers face three primary challenges:

- **Resource Overhead:** As a bundled browser instance, webviews are more memory-intensive than native rendering solutions.
- **Initialization:** Re-rendering the DOM upon opening the interface can cause delays or "white flashes," impacting perceived performance.[6]
- **System Sandboxing:** Webviews lack direct file system access, necessitating native workarounds for asset loading.

¹ The queue is a multi-producer, multi-consumer model because it allows thread-safe operations for both the VS Code backend and the audio plugin backend.

² Future versions of Cabbage 3 will explore the use of binary data streams to optimize data transfer.

Cabbage 3 addresses these issues through efficient, lock-free communication and native UI callbacks for file handling. While the industry is still evaluating webviews as a *de facto* standard, the successful deployment of this technology by companies like Tracktion, Native Instruments, and Arcade Inc.[7] proves that prioritizing development speed and UI flexibility over absolute resource minimization is becoming more attractive.

2.4 Cultivating the Cabbage Canvas

While Cabbage 3 provides a robust array of native UI elements, it also offers two distinct pathways for developers looking to break away from standard widgets and cultivate a custom canvas:

- **The Component Paradigm (Deep-Backend Custom Widgets):** This approach involves creating custom elements that inherit native lifecycle and data-binding behaviors directly from the Cabbage backend in the very same way native widgets do. The primary advantage of this approach is that these widgets remain completely accessible and configurable within the Cabbage UI editor.
- **The Application Paradigm (Standalone Interfaces via the JavaScript API):** This approach is for those who wish to replace the native canvas rendering entirely with a bespoke HTML/JS web application running inside the plugin WebView canvas.

When leveraging the Application Paradigm, developers utilize the Cabbage JavaScript API to establish a responsive, two-way communication system between the user interface and the backend. The core of this bidirectional pipeline relies on two primary functions:

- **Backend-to-UI Communication:** `Cabbage.addMessageListener(callback)`: Registers a listener for incoming messages from the Cabbage backend. This is the recommended way to handle all state updates originating from the DSP engine. It abstracts the two runtime environments so that the same UI code works seamlessly in the VS Code web panel and, more importantly, within audio plugins.³
- **UI-to-Backend Communication:** `Cabbage.sendControlData({ channel, value, gesture })`: Sends widget value changes back to the Cabbage backend. This is the primary API for capturing user interactions. Only channel value updates are permitted here; this ensures maximum performance and reduces the chance of data clogging the message queue.

Beyond these core communication hooks, the API provides several utility functions to handle essential plugin behaviors, including sending and receiving MIDI messages, triggering native file dialogs, and responding to plugin resizing callbacks. Additionally, a unique `sendChannelData(channel, data, vscode)` function provides developers with a means to bypass the parameter system entirely and update a Csound channel directly. Although deceptively simple, this lightweight API provides developers with everything required for efficient, robust frontend-to-backend communication.

3 AI-Assisted Development via Open-Source Ecosystems

Because Cabbage 3 uses well-defined JSON objects to describe UI elements, it is straightforward to leverage AI models to generate interfaces for any Csound orchestra, as well as to create custom widgets for bespoke tasks. While various free AI models are suitable, this paper employs MiniMax via the open-source OpenCode AI agent. Although primarily a terminal-based assistant, OpenCode offers integration packages for several IDEs. Since Cabbage 3 operates directly within VS Code, the OpenCode extension was utilized to maintain a unified, single-IDE development workflow for these examples.

3.1 Rapid Prototyping with Existing Cabbage Widgets

Cabbage 3 provides a hyper-focused, LLM-optimized JSON reference that serves as a canonical source of truth for UI generation. This reference file contains concise definitions for all native widgets that

³ All UI updates should come from the backend, not the frontend.

ship with the framework. AI models can inject this guide directly into their context window to eliminate hallucinations and ensure generated layout code strictly adheres to the Cabbage 3 specification. In empirical testing across various open-source LLMs, this reference-guided approach consistently yielded syntactically correct, functional UI definitions on the first attempt, provided the initial prompt was sufficiently well-defined.

3.2 Development of Custom Cabbage Widgets

To develop a new custom widget, a user simply invokes the **Create New Custom Widget** command from the VS Code command palette. This automatically generates a boilerplate template class containing the placeholder functions required to interact with the Cabbage backend. Because this boilerplate enforces a predictable, rigid structure, open-source AI models can reliably fill in the implementation details. All custom widgets created via this workflow must implement the following core components:

- **A properties object:** A JSON object containing properties exposed to, and accessible through, the native UI element inspectors in VS Code.
- **Event listeners:** Handlers designed to manage incoming messages originating from both the VS Code extension and the audio plugin's runtime WebView.
- **An inner HTML method:** A `getInnerHTML()` method used to generate and inject the initial DOM elements into the view.
- **Canvas methods:** If the widget requires pixel-level drawing, `createCanvas()` and `updateCanvas()` methods are implemented. These are automatically invoked by the system's `WidgetManager` to handle the embedded graphics context.
- **Reactive properties (Optional):** If a widget needs to actively listen for real-time updates to its properties, developers can wrap the main properties object (`this.props`) using `CabbageUtils.createReactiveProps()`.

3.3 Creating Entirely New Interfaces

Building entirely new user interfaces completely free of native Cabbage layout rendering is achieved by leveraging the standalone JavaScript API. The most efficient way to initiate this workflow is by executing the **Create Vanilla VST3...** command from the VS Code command palette. This action exports a functional plugin binary and automatically scaffolds a ready-to-use resource folder alongside it containing the following components:

- **A cabbage/ folder:** Contains the core `cabbage.js` framework file along with its supporting internal libraries.
- **A starter index.html:** A boilerplate entry point pre-wired to communicate directly with the Cabbage JavaScript API.⁴
- **A cabbage.project.json file:** A project configuration file pre-configured specifically for local file execution mode.

From within the newly created project resource directory, developers can initialize a modern web framework project of choice using standard build tools:

```
# Svelte / Vite setup
npm create vite@latest MyPlugin -- --template svelte
cd MyPlugin && npm install

# React / Vite setup
npm create vite@latest MyPlugin -- --template react
cd MyPlugin && npm install
```

The `cabbage.project.json` file can be configured to point directly to a local development server during the active design phase, or switched back to the compiled static assets for final distribution. This configuration file provides a highly flexible mechanism for testing custom interfaces alongside the running Csound instrument entirely within the unified VS Code environment.

⁴ This default file can be safely removed or bypassed if using a modern web build tool to bundle and generate a user-defined `index.html`.

4 Discussion: The Relevancy of Cabbage 3 in an AI-Driven World

As LLMs make generic code generation effortless, the relevance of specialized audio frameworks is frequently questioned. Cabbage 3 remains uniquely relevant because it establishes a highly efficient division of labor: human intuition drives the digital signal processing (DSP) via hand-written Csound, while machine automation scaffolds the user interface. The primary bottleneck in plugin development is rarely the DSP; it is the tedious process of designing, positioning, and binding GUI elements to audio parameters. By leveraging open-source models like MiniMax via OpenCode, a developer can offload layout generation to AI. Cabbage 3 acts as the perfect host for this symbiosis, keeping the human firmly in control of the high-performance audio thread while using AI to vaporize frontend development friction.

Furthermore, AI-assisted development is heavily constrained by LLM context windows and cognitive drift. Tasking an AI with generating both real-time audio logic and complex frontend code simultaneously drastically increases token consumption and the probability of catastrophic code hallucinations. Because Cabbage 3 cleanly decouples the frontend canvas from the audio engine, the AI's responsibilities are heavily minimized. The developer only needs to provide the model with a set of Csound channel mappings and Cabbage 3's optimized, lightweight JSON reference. Because this technical surface area is so small, open-source models can reliably output flawless UI layouts or lean two-way API bindings on the first attempt. Cabbage 3 ultimately transforms from a traditional framework into a high-leverage platform that amplifies human DSP expertise through targeted AI automation.

References

1. S. Overly, "The Cost of Intelligence: Structural Shifts and Subscription Realities in Commercial LLM Deployments," *Journal of AI Economic Perspectives*, vol. 14, no. 2, pp. 102–115, 2025.
2. V. Lazzarini, J. ffitc, S. Yi, J. Heintz, Ø. Brandtsegg, and I. McCurdy, *Csound: A Sound and Music Computing System*, Berlin, Germany: Springer, 2016.
3. Free-Audio Consortium, *CLAP (CLever Audio Plug-in) Specification: Core ABI and Extensions*, Version 1.2.8, 2026. [Online]. Available: <https://github.com/free-audio/clap>
4. R. Bencina, "Interfacing Real-Time Audio and File I/O," in *Proceedings of the 2014 Australasian Computer Music Conference (ACMC 2014)*, Melbourne, Australia, 2014, pp. 17–23.
5. JUCE Community, "WebView vs JUCE graphics," *JUCE Developer Forum*, May 2024. [Online Thread]. Available: <https://forum.juce.com/t/webview-vs-juce-graphics/61156>
6. JUCE Community, "Web view start up UI issue," *JUCE Developer Forum*, Mar. 2026. [Online Thread]. Available: <https://forum.juce.com/t/web-view-start-up-ui-issue/68561>
7. A. Carabott, "High Performance WebView User Interfaces: Getting the Most Out of the 2D Canvas and DOM," presented at the *Audio Developer Conference (ADC 2024)*, Bristol, UK, Nov. 2024. [Online Video]. Available: <https://conference.audio.dev/session/2024/high-performance-webview-user-interfaces/>
8. J. Manchanda, L. Boettcher, M. Westphalen, and J. Jasser, "The Open Source Advantage in Large Language Models (LLMs)," *arXiv preprint arXiv:2412.12004v3*, 2025.